Examen Primera Convocatoria, Franja Horaria Noche

```
# Your imports HERE !!!!
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

01 - 1.5pt

Escribir una función que identifique si un número entero dado es un número perfecto (la suma de sus divisores propios es igual al número mismo). Usando plain Python, sin librerías

```
# Your solution HERE !!!!
def is_perfect(n):
    contador = 0
    for i in range(1,n):
        if n%i == 0:
            contador+=i
    return n == contador

# Test Use Case
source_array = [233, 6, 5661, 348, 28, 901, 496, 1200, 8128, 2334]
expected_output = [False, True, False, False, True, False, True,
False, True, False]

output = [is_perfect(n) for n in source_array] # This calls your
function is_perfect

display(expected_output)
display(output)

assert(expected_output == output) # This will fail if the list is
wrong
```

```
[False, True, False, False, True, False, True, False, True, False]
```

```
[False, True, False, False, True, False, True, False, True, False]
```

02 - 1.5pt

- Crear un array de NumPy de 1 dimensión con 50 elementos, y enteros aleatorios entre 1 y 100
- Crear un histograma para ese conjunto de datos agrupando por rangos de números de 10 en 10

- Mostrar la linea de densidad
- Mostrar la media y la mediana como líneas verticales en el gráfico

```
# Your solution HERE !!!!
data = np.random.randint(1, 101, size=50)
print(data)

# Histograma
plt.hist(data, bins=10, edgecolor='black', color='orange', alpha=0.6,
density=True)

#Linea de densidad con Seaborn
sns.kdeplot(data, color='green', linewidth=2, label='Línea de
densidad')

# Líneas verticales para la media y la mediana
mean = np.mean(data)
median = np.median(data)
plt.axvline(mean, color='red', linestyle='--', label=f'Media:
{mean:.2f}')
plt.axvline(median, color='blue', linestyle='--', label=f'Mediana:
{median:.2f}')

#Configuraciones Extras
plt.title('Histograma')
plt.xlabel('Rango de números')
plt.ylabel('Densidad')
plt.legend()
plt.grid(alpha=0.4)
plt.show()
```

03 - 1.5pt

- Crear una matriz cuadrada 6x6 con valores aleatorios enteros en el rango [-10,10]
- Restar a cada elemento de dicha matriz la media de su correspondiente columna (dicha media redondeada hacia el entero superior más cercano)
- Sustituir los valores negativos múltiplos de 3 por Nan
- Eliminar aquellas columnas que tengan 2 o más celdas con valores Nan

```
#Matriz aleatoria
df = pd.DataFrame(np.random.randint(-10, 11,(6,6)))
display(df)
```

	0	1	2	3	4	5
0	-5	4	7	7	7	-8
1	1	9	0	-6	-2	2
2	-4	-7	-3	-8	-6	-9
3	-3	9	-6	-4	10	-10
4	5	-8	2	0	1	1
5	0	10	-9	-8	3	3

```python
# Restar a cada elemento de dicha matriz la media de su correspondiente columna (dicha media redondeada hacia el entero superior más cercano)
df2 = df.mean(axis=0)
df2 = np.ceil(df2)
data = df - df2
display(data)

# Sustituir los valores negativos múltiplos de 3 por Nan
data = data.mask((data < 0) & (data % 3 == 0), np.nan)
display(data)

#Eliminar aquellas columnas que tengan 2 o más celdas con valores Nan
data = data.loc[:, data.isna().sum() < 2] #localizar todas las filas y solo las columnas que cumplan la condicion
display(data)
```

	0	1	2	3	4	5
0	-4.0	1.0	8.0	10.0	4.0	-5.0
1	2.0	6.0	1.0	-3.0	-5.0	5.0
2	-3.0	-10.0	-2.0	-5.0	-9.0	-6.0
3	-2.0	6.0	-5.0	-1.0	7.0	-7.0
4	6.0	-11.0	3.0	3.0	-2.0	4.0
5	1.0	7.0	-8.0	-5.0	0.0	6.0

	0	1	2	3	4	5
0	-4.0	1.0	8.0	10.0	4.0	-5.0
1	2.0	6.0	1.0	NaN	-5.0	5.0
2	NaN	-10.0	-2.0	-5.0	NaN	NaN
3	-2.0	6.0	-5.0	-1.0	7.0	-7.0
4	6.0	-11.0	3.0	3.0	-2.0	4.0
5	1.0	7.0	-8.0	-5.0	0.0	6.0

	0	1	2	3	4	5
0	-4.0	1.0	8.0	10.0	4.0	-5.0
1	2.0	6.0	1.0	NaN	-5.0	5.0
2	NaN	-10.0	-2.0	-5.0	NaN	NaN
3	-2.0	6.0	-5.0	-1.0	7.0	-7.0
4	6.0	-11.0	3.0	3.0	-2.0	4.0
5	1.0	7.0	-8.0	-5.0	0.0	6.0

04 - 1.5pt

- Escribir un generador que genere todos los factores de un número. Un factor es un número natural que puede dividir exactamente a otro número.

```python
# Your solution HERE !!!!
def factor_gen(number):
    for i in range(1, number + 1):
        if number % i == 0:  # Verifica si i es un factor de n
            yield i  # Si es un factor, lo devuelve
```

```
# Test Use Case
number = 1000
expected_output = [1, 2, 4, 5, 8, 10, 20, 25, 40, 50, 100, 125, 200,
250, 500, 1000]

gen = factor_gen(number) # This calls your function factor_gen
output = [next(gen) for n in range(len(expected_output))]

display(expected_output)
display(output)

assert(expected_output == output)  # This will fail if the element is
not found correctly
```

```
[1, 2, 4, 5, 8, 10, 20, 25, 40, 50, 100, 125, 200, 250, 500, 1000]
```

```
[1, 2, 4, 5, 8, 10, 20, 25, 40, 50, 100, 125, 200, 250, 500, 1000]
```

05 - 4pt

Se dispone de dos conjuntos de datos, ambos datasets se adjuntan en la pregunta del examen

- 01MIAR_aircraft.csv: incidentes de avión.
- 01MIAR_airports.csv: aeropuertos de todo el mundo, con el nombre y país de cada aeropuerto.

1. Carga el fichero 01MIAR_aircraft.csv en un dataframe de Pandas y:
   - A. Muestra las seis primeras filas.
   - B. Calcula el porcentaje de valores nulos por columna.
   - C. Muestra en una figura, con un tamaño de 8 de ancho y 4 de alto, una gráfica de tipo puntos (scatter), la localización de los incidentes en base a la longitud y latitud.

     Pista, descarta del df original los registros cuyos valores de latitud y longitud no se encuentren en el rango de coordenadas esféricas, siendo la longitud valida en el rango [-180, 180], y la latitud valida en el rango [-90, 90].
   - D. Muestra en una figura un conteo de incidentes sobre la columna 'AIRCRAFT' por año, en el gráfico debe aparecer una columna por cada 'DAMAGE_LEVEL' distinto en el dataframe. La gráfica debe ser de tipo linea con diferentes subplots.

     Pista, agrupa primero y aplique una gráfica de relación.
2. Carga el fichero 01MIAR_airports.csv en un dataframe de Pandas y:
   - A. Muestra el número de filas y columnas
   - B. Cambia los valores a minúsculas de columna de nombre de aeropuerto en este dataframe y en el del apartado anterior.

- C. Con el propósito de disponer del país de los aeropuertos, junta ambos dataframes para obtener un nuevo dataframe con todos los registros de incidentes de avión y los países de los aeropuertos. Como clave, utilizar el nombre del aeropuerto modificado en el anterior apartado.
- D. Como resultado se debe disponer un dataframe con 254222 registros.
- E. Muestra el número de registros que no disponen de dato en el atributo "iso_country".
- F. Elimina todos los registros que que tengan nulos alguna de las columnas 'DAMAGE_LEVEL' y 'iso_country'.
- G. Sobre el resultado anterior, crea una tabla contando el número de incidentes por país, en las filas deben aparecer todos los paises posibles y en las columnas debe de estar el tipo de 'DAMAGE_LEVEL'.

```
# Your solution HERE !!!!
#1
#A. Muestra las seis primeras filas.
data_aircraft = pd.read_csv('01MIAR_aircraft.csv', delimiter=',')
data_aircraft.head(6)
```

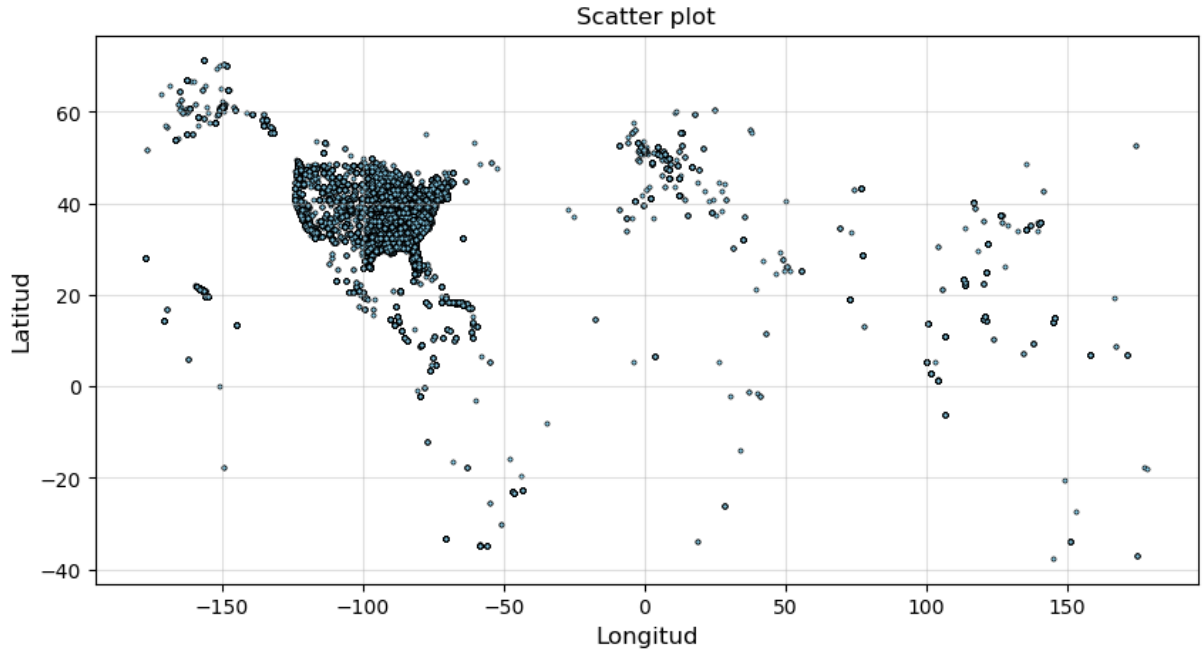
```
  INCIDENT_DATE                       AIRPORT  LATITUDE  LONGITUDE
AIRCRAFT  \
0     6/22/1996              SACRAMENTO INTL  38.69542 -121.59077  B-
737-300
1     6/26/1996          DENVER INTL AIRPORT  39.85841 -104.66700  B-
737-300
2      7/1/1996              EPPLEY AIRFIELD  41.30252  -95.89417  B-
757-200
3      7/1/1996  WASHINGTON DULLES INTL ARPT  38.94453  -77.45581
A-320
4      7/1/1996              LA GUARDIA ARPT  40.77724  -73.87261
A-320
5      5/6/1991             SAN ANTONIO INTL  29.53369  -98.46978  B-
727-100

  DAMAGE_LEVEL
0          NaN
1          NaN
2            N
3            N
4            M
5            N
```

```
#B. Calcula el porcentaje de valores nulos por columna.
nan_total= data_aircraft.isnull().sum()
nan_percent = (nan_total/data_aircraft.count())* 100
print(nan_percent)
```

```
INCIDENT_DATE     0.000000
AIRPORT           0.000000
LATITUDE         14.014899
```

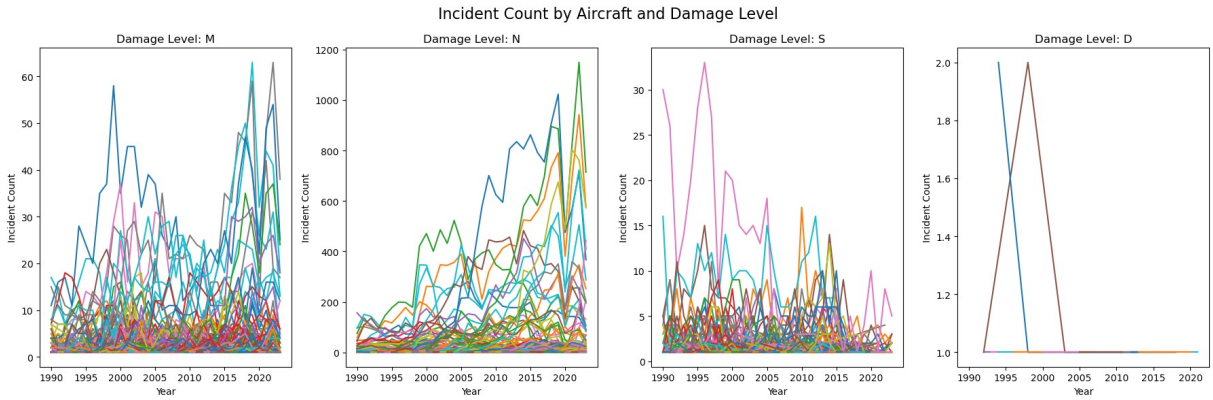
```
LONGITUDE         14.015349
AIRCRAFT           0.000000
DAMAGE_LEVEL      54.184128
dtype: float64
```

```
#C. Muestra en una figura, con un tamaño de 8 de ancho y 4 de alto,
una gráfica de tipo puntos (scatter), la localización de los
incidentes en base a la longitud y latitud
df = data_aircraft
df_filtrado = df[(df['LATITUDE'] >= -90) & (df['LATITUDE'] <= 90) &
                 (df['LONGITUDE'] >= -180) & (df['LONGITUDE'] <= 180)]

longitud = df_filtrado["LONGITUDE"]
latitud = df_filtrado["LATITUDE"]

plt.figure(figsize=(10,5))
plt.scatter(longitud, latitud, s=5, edgecolor='black', linewidth=0.5,
color='skyblue')

plt.title("Scatter plot");
plt.xlabel('Longitud', fontsize=12)
plt.ylabel('Latitud', fontsize=12)
plt.grid(alpha=0.4)
plt.show()
```




```
# D. Muestra en una figura un conteo de incidentes sobre la columna
'AIRCRAFT' por año, en el gráfico debe aparecer una columna por cada
'DAMAGE_LEVEL' distinto en el dataframe. La gráfica debe ser de tipo
```

```
linea con diferentes subplots
df["INCIDENT_DATE"] = pd.to_datetime(df["INCIDENT_DATE"])
df["YEAR"]= df["INCIDENT_DATE"].dt.year

grouped = df.groupby(["YEAR", "AIRCRAFT",
"DAMAGE_LEVEL"]).size().reset_index(name="COUNT")

damage_levels = grouped["DAMAGE_LEVEL"].unique()

fig, axes = plt.subplots(1,len(damage_levels), figsize=(18, 6))

# Iterar por cada nivel de daño y graficar
for ax, level in zip(axes, damage_levels):
    subplot = grouped[grouped["DAMAGE_LEVEL"] == level]
    for aircraft in subplot["AIRCRAFT"].unique():
        aircraft_data = subplot[subplot["AIRCRAFT"] == aircraft]
        ax.plot(aircraft_data["YEAR"], aircraft_data["COUNT"])

    ax.set_title(f"Damage Level: {level}")
    ax.set_ylabel("Incident Count")
    ax.set_xlabel("Year")

fig.suptitle("Incident Count by Aircraft and Damage Level",
fontsize=16)
plt.tight_layout()
plt.show()
```




```
# 2
data_airports = pd.read_csv("01MIAR_airports.csv", delimiter=',')
data_airports.head(5)
```

	id	ident	type	name	latitude_deg	\
0	6523	00A	heliport	Total RF Heliport	40.070985	
1	323361	00AA	small_airport	Aero B Ranch Airport	38.704022	
2	6524	00AK	small_airport	Lowell Field	59.947733	
3	6525	00AL	small_airport	Epps Airpark	34.864799	
4	506791	00AN	small_airport	Katmai Lodge Airport	59.093287	

```
   longitude_deg  elevation_ft continent iso_country iso_region
municipality  \
0     -74.933689          11.0       NaN          US      US-PA
Bensalem
1    -101.473911        3435.0       NaN          US      US-KS
Leoti
2    -151.692524         450.0       NaN          US      US-AK
Anchor Point
3     -86.770302         820.0       NaN          US      US-AL
Harvest
4    -156.456699          80.0       NaN          US      US-AK   King
Salmon

  scheduled_service gps_code iata_code local_code  \
0                no     K00A       NaN        00A
1                no     00AA       NaN       00AA
2                no     00AK       NaN       00AK
3                no     00AL       NaN       00AL
4                no     00AN       NaN       00AN

                                          home_link wikipedia_link
keywords
0  https://www.penndot.pa.gov/TravelInPA/airports...            NaN
NaN
1                                                NaN            NaN
NaN
2                                                NaN            NaN
NaN
3                                                NaN            NaN
NaN
4                                                NaN            NaN
NaN
```

```
# A. Muestra el número de filas y columnas
display(data_airports.shape)
print("filas:", data_airports.shape[0])
print("columnas:", data_airports.shape[1])
```

```
(77458, 18)

filas: 77458
columnas: 18
```

```
# B. Cambia los valores a minúsculas de columna de nombre de
aeropuerto en este dataframe y en el del apartado anterior.
data_airports["name"] = data_airports["name"].str.lower()
data_aircraft["AIRPORT"] = data_aircraft["AIRPORT"].str.lower()

# C. Con el propósito de disponer del país de los aeropuertos, junta
ambos dataframes para obtener un nuevo dataframe con todos los
```

```
registros de incidentes de avión y los países de los aeropuertos. Como
clave, utilizar el nombre del aeropuerto modificado en el anterior
apartado.
data_merged = pd.merge(data_aircraft, data_airports,
left_on="AIRPORT", right_on="name", how="left")
display(data_merged)
```

```
       INCIDENT_DATE                          AIRPORT  LATITUDE
LONGITUDE  \
0         1996-06-22                sacramento intl  38.69542 -
121.59077
1         1996-06-26             denver intl airport  39.85841 -
104.66700
2         1996-07-01                  eppley airfield  41.30252  -
95.89417
3         1996-07-01      washington dulles intl arpt  38.94453  -
77.45581
4         1996-07-01                  la guardia arpt  40.77724  -
73.87261
...              ...                              ...       ...
...
289720    2023-08-28                          unknown       NaN
NaN
289721    2023-08-28                          unknown       NaN
NaN
289722    2023-08-28  detroit metro wayne county arpt  42.21206  -
83.34884
289723    2023-08-28          pocatello regional arpt  42.91131 -
112.59586
289724    2023-08-28       rick husband amarillo intl  35.21937 -
101.70593

              AIRCRAFT DAMAGE_LEVEL  YEAR      id ident           type
...  \
0          B-737-300          NaN  1996     NaN   NaN            NaN
...
1          B-737-300          NaN  1996     NaN   NaN            NaN
...
2          B-757-200            N  1996  3751.0  KOMA  large_airport
...
3              A-320            N  1996     NaN   NaN            NaN
...
4              A-320            M  1996     NaN   NaN            NaN
...
...              ...          ...   ...     ...   ...            ...
...
289720         A-320            N  2023     NaN   NaN            NaN
...
289721  GULFSTREAM IV            N  2023     NaN   NaN            NaN
...
```

```
289722         CRJ900            N  2023     NaN   NaN            NaN
...
289723    BE-200 KING            N  2023     NaN   NaN            NaN
...
289724        UNKNOWN          NaN  2023     NaN   NaN            NaN
...

       iso_country  iso_region  municipality  scheduled_service
gps_code  \
0              NaN         NaN           NaN                NaN
NaN
1              NaN         NaN           NaN                NaN
NaN
2               US       US-NE         Omaha                yes
KOMA
3              NaN         NaN           NaN                NaN
NaN
4              NaN         NaN           NaN                NaN
NaN
...            ...         ...           ...                ...      .
..
289720         NaN         NaN           NaN                NaN
NaN
289721         NaN         NaN           NaN                NaN
NaN
289722         NaN         NaN           NaN                NaN
NaN
289723         NaN         NaN           NaN                NaN
NaN
289724         NaN         NaN           NaN                NaN
NaN

       iata_code local_code home_link  \
0            NaN        NaN       NaN
1            NaN        NaN       NaN
2            OMA        OMA       NaN
3            NaN        NaN       NaN
4            NaN        NaN       NaN
...          ...        ...       ...
289720       NaN        NaN       NaN
289721       NaN        NaN       NaN
289722       NaN        NaN       NaN
289723       NaN        NaN       NaN
289724       NaN        NaN       NaN

                                       wikipedia_link keywords
0                                                 NaN      NaN
1                                                 NaN      NaN
2      https://en.wikipedia.org/wiki/Eppley_Airfield      NaN
3                                                 NaN      NaN
```

```
4                                                     NaN      NaN
...                                                   ...      ...
289720                                                NaN      NaN
289721                                                NaN      NaN
289722                                                NaN      NaN
289723                                                NaN      NaN
289724                                                NaN      NaN

[289725 rows x 25 columns]
```

```
# D. Como resultado se debe disponer un dataframe con 254222
registros.

# E. Muestra el número de registros que no disponen de dato en el
atributo "iso_country".
print(data_merged["iso_country"].isna().sum())
```

```
280828
```

```
# F. Elimina todos los registros que que tengan nulos alguna de las
columnas 'DAMAGE_LEVEL' y 'iso_country'.
data_clean = data_merged.dropna(subset=['DAMAGE_LEVEL',
'iso_country'])
data_clean.head(5)
```

```
    INCIDENT_DATE                    AIRPORT  LATITUDE  LONGITUDE
AIRCRAFT  \
2      1996-07-01           eppley airfield  41.30252  -95.89417  B-
757-200
32     1993-09-19        manchester airport  42.93452  -71.43706
BE-33
99     1990-10-26  cape cod gateway airport  41.66934  -70.28036
C-402
132    1990-05-12           eppley airfield  41.30252  -95.89417  B-
727-200
179    1990-07-11           eppley airfield  41.30252  -95.89417
DC-9-30

    DAMAGE_LEVEL  YEAR      id ident            type  ... iso_country
\
2              N  1996  3751.0  KOMA   large_airport  ...          US

32             S  1993  2398.0  EGCC   large_airport  ...          GB

99             M  1990  3600.0  KHYA  medium_airport  ...          US

132            N  1990  3751.0  KOMA   large_airport  ...          US

179            N  1990  3751.0  KOMA   large_airport  ...          US
```

```
     iso_region  municipality  scheduled_service gps_code iata_code  \
2         US-NE         Omaha                yes     KOMA        OMA
32       GB-ENG    Manchester                yes     EGCC        MAN
99        US-MA       Hyannis                yes     KHYA        HYA
132       US-NE         Omaha                yes     KOMA        OMA
179       US-NE         Omaha                yes     KOMA        OMA

    local_code                              home_link  \
2          OMA                                    NaN
32         NaN  http://www.manchesterairport.co.uk/
99         HYA                                    NaN
132        OMA                                    NaN
179        OMA                                    NaN

                                        wikipedia_link  \
2        https://en.wikipedia.org/wiki/Eppley_Airfield
32    https://en.wikipedia.org/wiki/Manchester_Airport
99   https://en.wikipedia.org/wiki/Barnstable_Munic...
132      https://en.wikipedia.org/wiki/Eppley_Airfield
179      https://en.wikipedia.org/wiki/Eppley_Airfield

                          keywords
2                              NaN
32   Ringway Airport, RAF Ringway
99   Barnstable, Boardman Polando
132                            NaN
179                            NaN

[5 rows x 25 columns]
```

```
# G. Sobre el resultado anterior, crea una tabla contando el número de
incidentes por país, en las filas deben aparecer todos los paises
posibles y en las columnas debe de estar el tipo de 'DAMAGE_LEVEL'.
grouped = data_clean.groupby(["iso_country",
'DAMAGE_LEVEL']).size().unstack(fill_value=0)
display(grouped)
```

```
DAMAGE_LEVEL  D    M     N    S
iso_country
BR            0    0     1    0
BZ            0    0     1    0
CA            0    6     4    1
DM            0    0     1    0
ES            0    1     7    0
FM            0    0     7    0
GB            0   14   315    3
GY            0    6     4    1
IE            0    0     2    0
IN            0    0     1    0
IT            0    0     1    0
```

```
MX            0    0     1    0
PT            0    0     1    0
SA            0    0     1    0
SG            0    0    51    1
TR            0    0     1    0
UM            0   12    73    0
US            8  634  4051  182
VC            0    1     0    0
VI            0    0     1    0
```